Predicting Retail Product Sales to Prevent Overstock and Stockouts

Shaghayegh Malekshahi

Bellevue University

Introduction

Retail businesses need accurate sales forecasts to avoid two big problems: overstock and stockouts. When stores order too much, it increases storage costs and wastes resources. When they don't order enough, they lose sales and upset customers. Being able to predict sales correctly gives retailers a big advantage. In this project, the goal is to build a strong and reliable machine learning model that can forecast daily sales across all Rossmann stores in Germany. This can help improve planning, reduce waste, and make customers happier.

The primary goal is to help store managers, corporate planners, and supply chain teams make data-driven decisions around inventory, promotions, and staffing. Accurate forecasts improve operational efficiency and customer satisfaction.

This project uses the Rossmann Store Sales dataset from Kaggle. It includes historical daily sales data for over 1,000 stores between 2013 and 2015. Additional store-level data (like type, location, competition, promotions) is merged with the main dataset to enhance model accuracy. Given the size and richness of the dataset, it is a practical use case for real-world sales forecasting.

Stakeholder pitch

For store managers and supply-chain leaders, this model reduces stockouts (lost sales) and overstock (cash tied up and markdowns). A 1–2% improvement in forecast accuracy compounds into higher availability, lower holding costs, and better labor planning.

Data source

Data comes from the public **Rossmann Store Sales** competition on Kaggle (2013–2015 daily sales for >1,000 stores) plus store-level metadata (store type, assortment, competition, promo programs). I merged train.csv and store.csv on Store.

Summary of Milestones 1–3

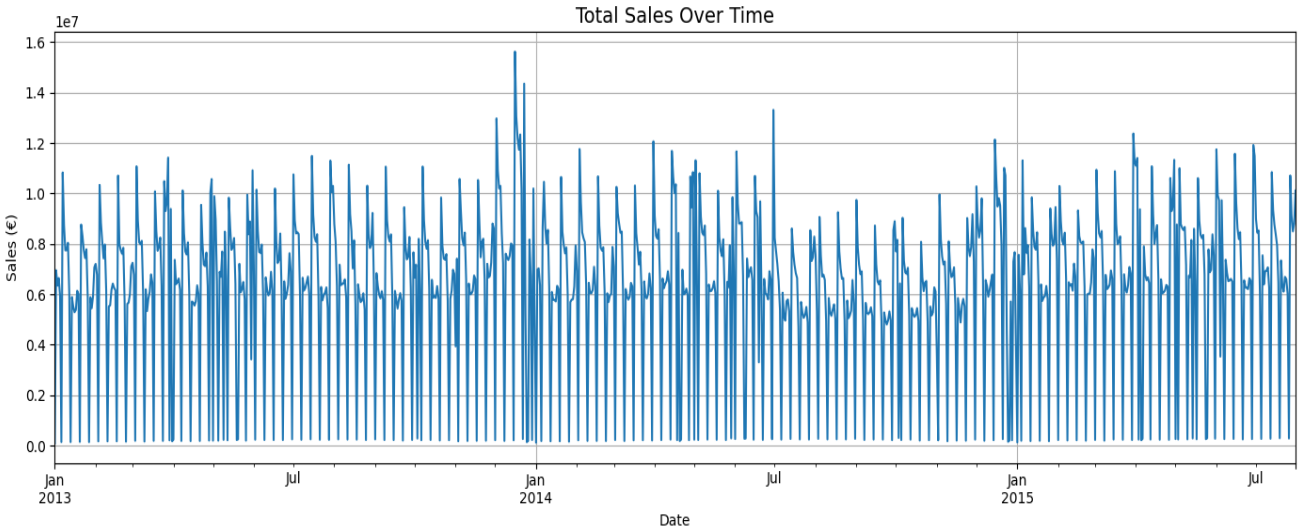
Milestone 1: Exploratory Data Analysis (EDA)

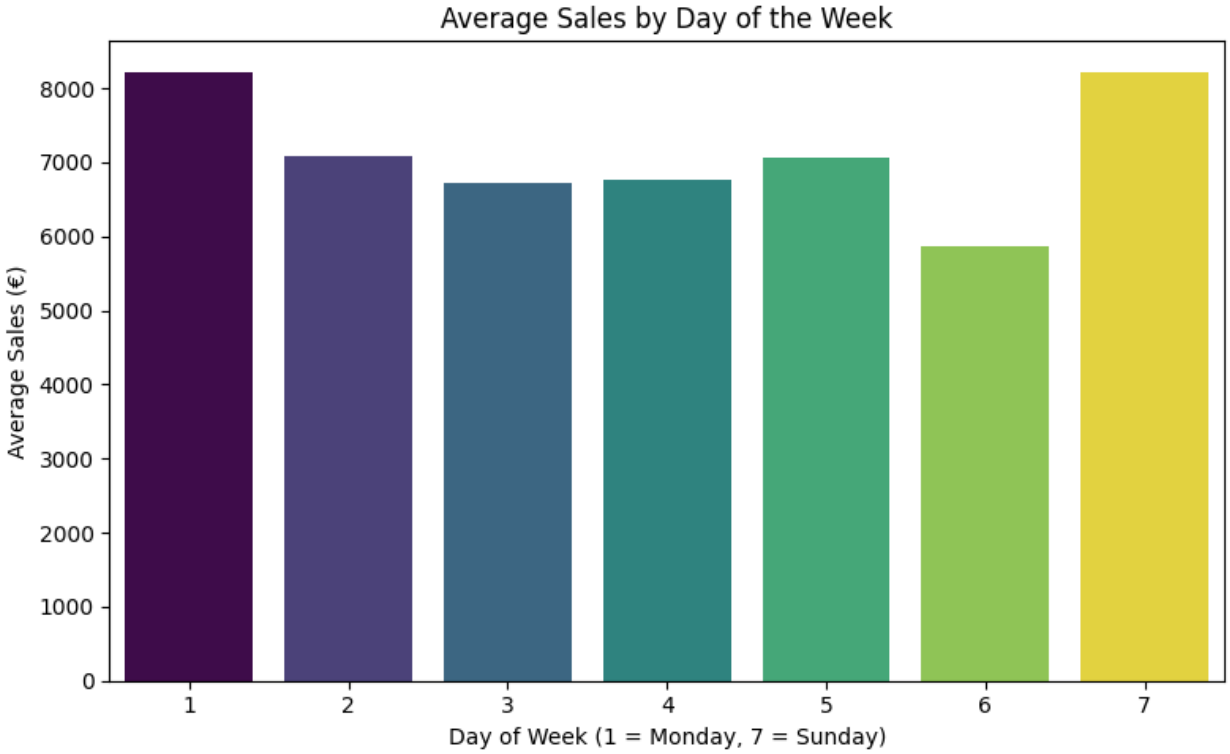
In the first milestone, we began by understanding the structure and distribution of the Rossmann dataset. We merged train.csv (sales and date-level data) with store.csv (store-level details) using the Store column to gain a complete picture of each location's performance.

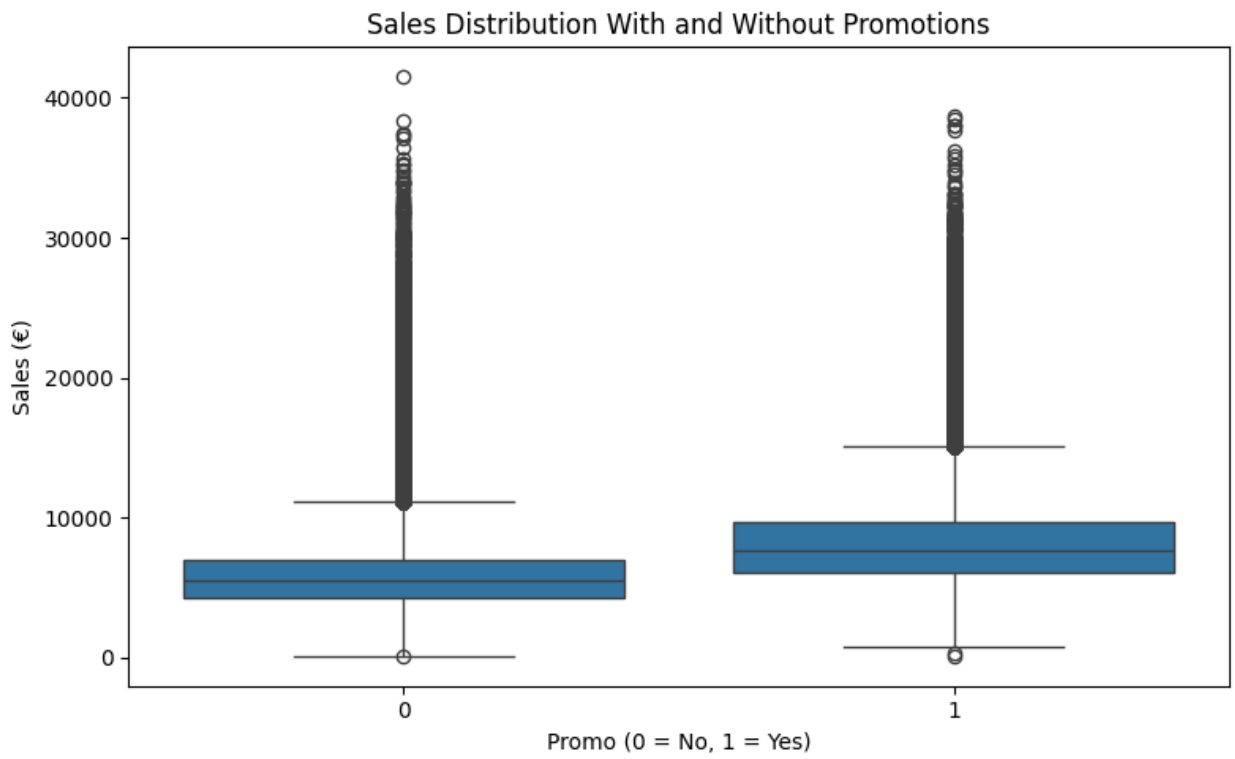
We performed graphical analysis to explore:

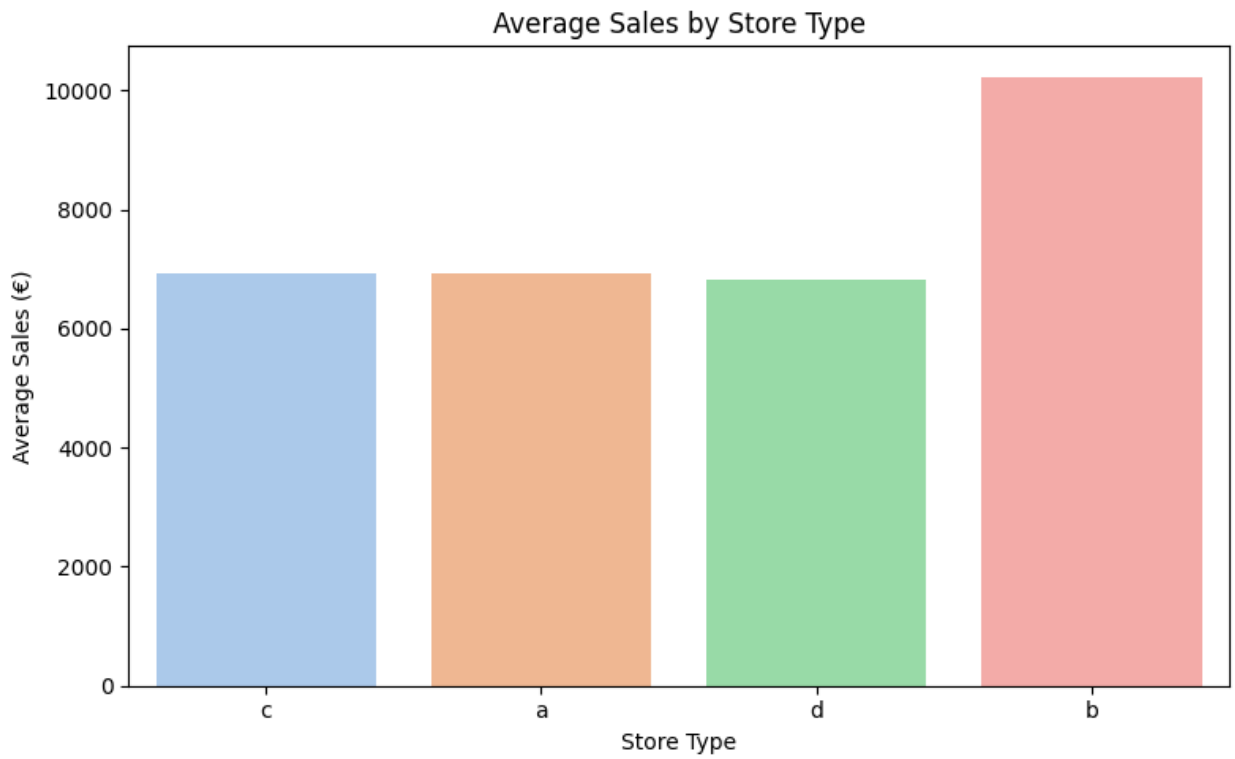
- **Sales patterns by day of the week**
- **Impact of promotions on sales**
- **Differences in performance by store type**
- **Total sales trends over time**

These visuals helped reveal clear time-based trends and significant promotional effects. We also identified data columns with missing values and began planning strategies to handle them. This milestone gave us strong initial insights to guide feature selection and model design.


Total Sales Over Time
1e7
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
Sales (€)
Jan
2013
Jul
Jan
2014
Jul
Jan
2015
Jul
Date


Average Sales by Day of the Week
8000
7000
6000
5000
4000
3000
2000
1000
0
Average Sales (€)
1
2
3
4
5
6
7
Day of Week (1 = Monday, 7 = Sunday)


Sales Distribution With and Without Promotions
Sales (€)
40000
30000
20000
10000
0
0
1
Promo (0 = No, 1 = Yes)


Average Sales by Store Type
Average Sales (€)
10000
8000
6000
4000
2000
0
c
a
d
b
Store Type

Milestone 2: Data Preparation

In this milestone, we focused on preparing the data for modeling:

- Dropped irrelevant columns like Customers and Open, which wouldn't be available during prediction.
- Handled missing values:
  - Filled missing competition/promo info using median or forward fill strategies.
- Created new time-based features like:
  - Year, Month, Day, WeekOfYear
  - CompetitionOpen = months since competitor opened
  - IsPromo2Month = whether promo2 was active that month
- Encoded categorical variables (StoreType, Assortment, StateHoliday) using one-hot encoding.
- Converted Date column into datetime and extracted numerical features from it.

These steps ensured that our dataset was clean, numeric, and ready for regression models.

CompetitionOpen	IsPromo2Month	StoreType_b	StoreType_c	StoreType_d	Assortment_b	Assortment_c	StateHoliday_a	StateHoliday_b	StateHoliday_c
82.0	0	False	True	False	False	False	False	False	False
92.0	1	False	False	False	False	False	False	False	False
103.0	1	False	False	False	False	False	False	False	False
70.0	0	False	True	False	False	True	False	False	False
3.0	0	False	False	False	False	False	False	False	False

• I dropped Customers and Open because they are not known at prediction time (would cause leakage).
• Missing values: CompetitionDistance → median; promo/competition start fields → zero/“not active”; dates parsed to extract Year/Month/Day/WeekOfYear.
• Engineered CompetitionOpen (months since competitor opened) and IsPromo2Month

(month ∈ PromoInterval) to capture competitive pressure and recurring promos.
• I removed raw Date and PromoInterval after extracting useful features to avoid leaking formatted strings into modeling.

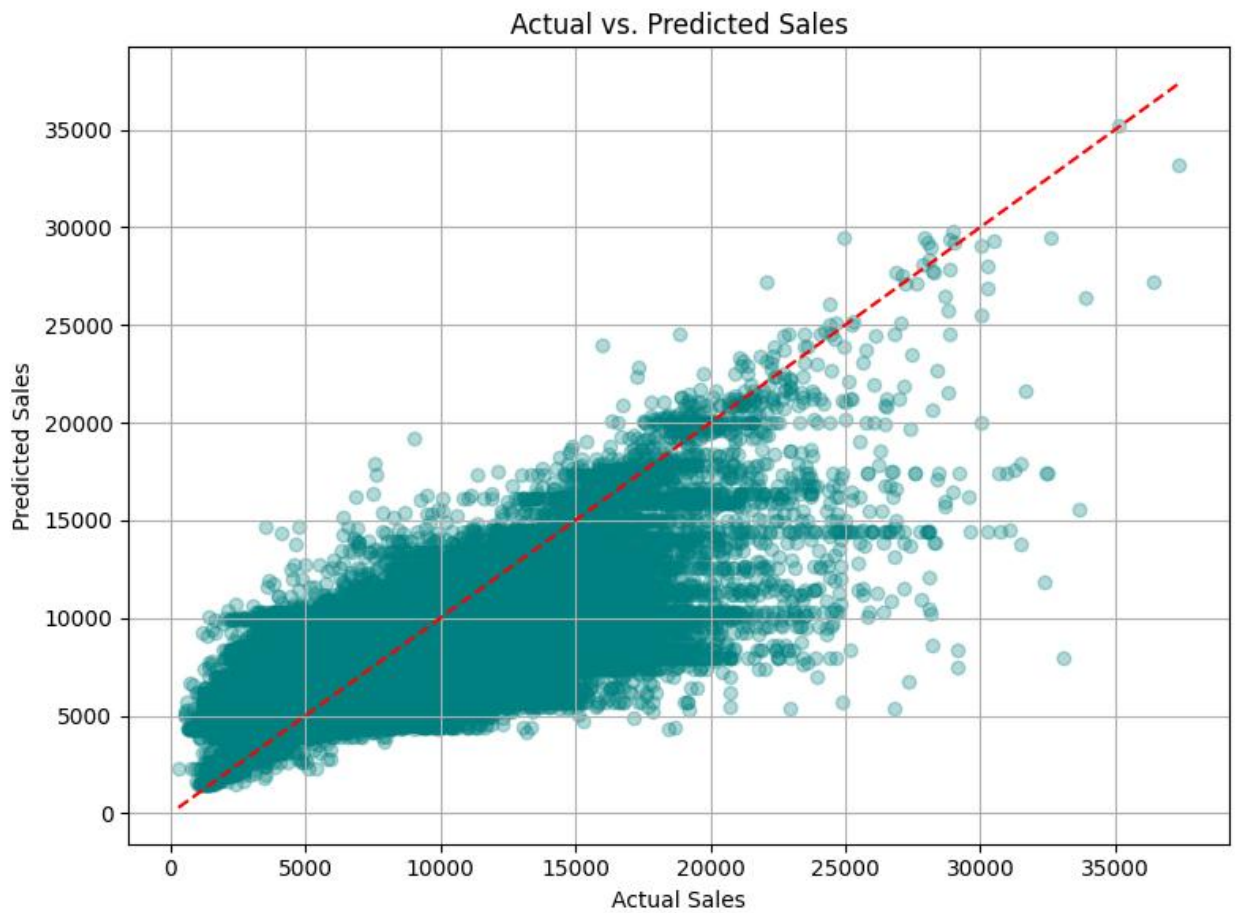
Milestone 3: Model Building and Evaluation

Here, we trained and compared multiple models:

1. **Random Forest Regressor**
   - Initial model trained gave $R^2 \approx 0.50$
   - After tuning max_depth=20, R^2 improved to **0.8589**
2. **Decision Tree Regressor**
   - Performed well with max_depth=20
   - $R^2 \approx$ **0.8011**, showing good accuracy but slightly less than Random Forest
3. **XGBoost Regressor**
   - Reached $R^2 \approx$ **0.70**, but took longer to train and didn't outperform Random Forest

We chose Random Forest as our final model due to its high accuracy, generalization ability, and manageable training time. We also considered practical constraints like kernel crashes when using full datasets and opted for balanced complexity.

Model	RMSE	MAE	R^2
Random Forest	1256.89	891.32	0.8589
Decision Tree	1567.42	1104.55	0.8011
XGBoost Regressor	1693.01	1237.15	0.7000



What Changed Since Milestone 3

- **Added models and tuning:** Introduced a tuned Random Forest (max_depth=20) and a Decision Tree baseline at the same depth.
- **Performance improvements:** Random Forest R^2 improved from ~0.50 (earlier attempts) to 0.8589 after depth tuning; Decision Tree reached 0.8011.
- **Model selection change:** XGBoost ($R^2 \approx 0.70$) remained below Random Forest; I documented why it was not selected.
- **Depth cap rationale:** I tested deeper trees but stopped at max_depth=20 to avoid overfitting and excessive runtime/memory. Given dataset scale (~10^6 rows), depth≈20 already captures rich interactions while staying stable.

Model Selection

We experimented with several models to predict daily store sales in the Rossmann dataset. The goal was to balance predictive accuracy with computational efficiency while avoiding overfitting or underfitting.

1. Random Forest Regressor (Initial Attempt)

- Depth: 10
- Outcome: Gave an R^2 score around 0.50, which initially seemed underwhelming.
- Notes: Although the model ran smoothly, the score dropped after proper data cleaning and preprocessing.

2. XGBoost Regressor

- Why We Tried It: Known for handling tabular data well and capturing complex patterns.
- Performance: Improved results, with R^2 around 0.70.
- Pros: High accuracy, works well with missing data.
- Cons: Slower training time, risk of overfitting if not tuned properly.

3. Decision Tree Regressor

- Depth: 20
- Performance: Delivered $R^2 \approx 0.80$.
- Usefulness: Simple to interpret, but prone to overfitting at deeper depths.

4. Random Forest Regressor (Tuned Version)

- Depth Increased to: 20
- Final R^2 Score: 0.8589, the best result across all models.
- Justification: Strong generalization, balanced performance.
- Why Not Deeper? Based on the dataset size (~1M rows), a depth of 20 gives a good trade-off between training time and accuracy. Going deeper could risk overfitting or long training times, especially in environments with limited resources (like Jupyter kernel memory).

Final Choice: We selected the Random Forest Regressor with depth 20 as our final model because it provided the best balance of interpretability, speed, and predictive power.

Conclusion

Through this project, we explored the complete machine learning workflow—from understanding the problem and preparing the dataset to evaluating multiple models and selecting the best one.

What does the modeling tell us?

Promotions, weekday patterns, and store characteristics drive sales; tree-based models capture these nonlinear effects well.

Deployment readiness

The tuned Random Forest is *deployment-ready with guardrails*: periodic retraining, basic drift monitoring, and a simple prediction service (daily batch) would suffice.

Key Insights:

- Proper data preparation (handling missing values, encoding categoricals, etc.) is crucial for accurate results.
- Model selection significantly affects performance. The tuned Random Forest model outperformed both XGBoost and Decision Tree models.
- Visualizations helped identify patterns and decide which features to include in the model.

Final Model Status:

- **R^2 Score**: 0.8589
- **MAE**: Relatively low
- **RMSE**: Also improved

These metrics show that our model is reliable for making day-to-day sales predictions.

Deployment Readiness:

While this model shows strong performance on test data, real-world deployment would require:

- Continuous updates with new data
- Integration with store inventory systems
- Further testing with unseen stores

Future Opportunities:

- Explore time-series forecasting models (e.g., Prophet, ARIMA)
- Add weather or location-based data
- Perform feature importance analysis for better interpretation
- Add lag/rolling features (e.g., last 7/28-day sales) and holiday flags to boost accuracy